

# Supplementary Material

Energy-Aware Deep Learning Compilation: Coding, Evidence and Reproducibility

Xincheng HE, Yan LIU

2026-09-23

The machine-readable version of this document and all data files are in the companion repository:

<https://github.com/itachiliu/Energy-Aware-Deep-Learning-Compilation>

The paper points to this repository as its supplementary repository.

## A. This document and the file index

All evidence beyond the main text is grouped by purpose into seven sets. Paths are given relative to the repository root.

- **S1, the 0/18 evidence:** `ecosystem/ecosystem-selection.md`;
- **S2, matrix counts:** `matrix/matrix-counts-4x4.md`, `matrix-counts-4x4.tex`, `matrix/count_matrix.py`;
- **S3, coding and agreement:** `coding/coding-121-final.csv`, the four first-round sheets `coding-sheet-A/B-v2.csv` and `-v3.csv`, the two codebooks `codebook-*-worked-examples.csv`, the controlled vocabularies `coding-rubric-v2.md` and `-v3.md`, the disagreement list `reconciliation-sheet`, the two reconciliation rounds `reconciliation-resolution-1-full.csv` and `-2.csv`, the final resolution table `reconciliation-resolution-final.txt`, the third-round adjudication package `adjudication-round3-*`, and the  $\kappa$ , merge and appendix-synchronisation scripts with their reports under `coding/kappa/`;
- **S4, historical audit:** `audit/appendix-matrix-code.csv`, `code_matrix.py`, `internal-audit.md`;
- **S5, independent spot check:** `audit/coding-audit-sample30.csv`, `coding-audit-protocol.md`, `audit_coding.py`;
- **S6, small-scale validation experiment:** `experiment/`, holding the protocol, the per-scan result tables, the environment snapshots, the job package and the plotting scripts; the per-run measurement logs are collected in `experiment/data/raw-measurement-logs.zip`;
- **S7, post-window tracking and environment:** `methodology/2026-preprint-tracking.md`, `experiment/ENV_REQUIREMENTS.md`;
- **Search and screening:** `methodology/prisma_counts.csv`; the per-database reconstructed queries are given in Appendix B of the paper.

## 1 S1. The 0/18 ecosystem table: systems and evidence carriers

The evidence hierarchy is paper > official documentation > public source code; entries that name a source location can be checked directly. The 0/18 scope is the 18 representative general-purpose or production-grade stacks selected in this survey (16 independent projects), with a time slice of 2026-05.

| #  | System / component | Paper or documentation evidence | Source / checkable point                                                                                    |
|----|--------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1  | TVM                | chen_tvm_2018                   | Open source; <code>ProgramMeasurer</code> in <code>src/auto_scheduler/measure.cc</code>                     |
| 2  | Ansor (TVM)        | zheng_ansor_2020                | Same as above; the cost model predicts latency by default                                                   |
| 3  | TVM Unity / Relax  | tvm_unity                       | Official repository and documentation (frozen at submission)                                                |
| 4  | TorchInductor      | ansel_pytorch_2024              | Official repository and documentation (frozen at submission)                                                |
| 5  | Triton             | triton                          | Official repository (frozen at submission)                                                                  |
| 6  | XLA                | openxla                         | <code>ProfileGuidedLatencyEstimator</code> ; GPU Speed-of-Light cost model                                  |
| 7  | IREE               | iree                            | Official repository and documentation (frozen at submission)                                                |
| 8  | TensorRT           | nvidia_tensorrt                 | Documentation only; closed source, peripheral components open                                               |
| 9  | TensorRT-LLM       | nvidia_trtllm                   | Open repository; execution depends on the closed TensorRT runtime                                           |
| 10 | OpenVINO           | intel_openvino                  | Official repository and documentation (frozen at submission)                                                |
| 11 | ONNX Runtime       | onnxruntime                     | Official repository and documentation (frozen at submission)                                                |
| 12 | TFLite / NNAPI     | google_tflite                   | Documentation only;                                                                                         |
| 13 | Core ML            | apple_coreml                    | <code>ANEURALNETWORKS_PREFER_LOW_POWER</code> ; Documentation only; <code>MLComputeUnits</code> enumeration |
| 14 | ExecuTorch         | executorch                      | Official repository and documentation (frozen at submission)                                                |
| 15 | MNN                | mnn                             | Official repository (frozen at submission)                                                                  |
| 16 | NCNN               | ncnn                            | Official repository (frozen at submission)                                                                  |
| 17 | Paddle Lite        | paddl-lite                      | Official repository (frozen at submission)                                                                  |
| 18 | MLC-LLM            | mlc-llm                         | Official repository (frozen at submission)                                                                  |

Traceable candidates that were not selected: serving-time continuous-batching runtimes such as vLLM (the decision subject is the serving system), 2026 preprints (MileStone, AutoPass, DualScale), and the Dynamo front end of `torch.compile` (its back end is TorchInductor).

## 2 S2. The $4 \times 4$ matrix counts and empty cells

Counting rule: only Appendix A samples whose granularity and mechanism both fall inside the four categories are counted ( $n = 68$ ); the coding comes from `coding-121-final.csv`, and empty-cell determination combines the Appendix coding with the representative works discussed in the text (counting script `matrix/count_matrix.py`).

| Granularity cell        | Analytical | Learned | AutoSearch | Agentic |
|-------------------------|------------|---------|------------|---------|
| Graph                   | 6          | 3       | 11         | 0       |
| Loop / operator         | 4          | 4       | 19         | 1       |
| Heterogeneous placement | 2          | 3       | 5          | 0       |
| Power state             | 4          | 3       | 3          | 0       |

Row totals: Graph 20, Loop/Op 28, Placement 10, Power 10.

Empty cells (Appendix count zero and no representative in the text): Placement  $\times$  Agentic and Power  $\times$  Agentic. Graph  $\times$  Agentic has a count of zero but is treated as non-empty because of a non-statistical representative discussed in the text (Agentic MLIR).

## 3 S3. Two-coder independent coding and agreement

The coded fields are category, granularity, mechanism and object type. Both coders completed them independently against a public controlled vocabulary; the manuals and worked examples are `coding/coding-rubric-v2.md`, `coding/coding-rubric-v3.md`, `codebook-v2-worked-examples.csv` and `codebook-v3-worked-examples.csv`. Units fixed by the worked-example sheets are excluded from the agreement statistics.

| Field       | $N$ | Agreement | Cohen's $\kappa$ |
|-------------|-----|-----------|------------------|
| Category    | 118 | 79.7%     | 0.77             |
| Granularity | 118 | 74.6%     | 0.66             |
| Mechanism   | 114 | 73.7%     | 0.65             |
| Object type | 118 | 89.8%     | 0.85             |

Mechanism and object type were coded independently a second time after the manual was extended; the first-round mechanism  $\kappa = 0.56$ , and that low value is precisely why the M1/M2 criteria were added to the manual and the field was recoded. The earlier boundary label set (core/bridging/frontier,  $\kappa = 0.46$ ) has been replaced by the object-type field. The complete scripts and reports are under `coding/kappa/`.

Independent coding produced 96 disagreeing units (category 24, granularity 30, mechanism 30, object type 12). The two coders first reconsidered the units one by one, which resolved 31; for the remaining 65 the procedure changed to first fixing a criterion and then applying it, in a third joint round that produced a resolution table covering all 96 units (`reconciliation-resolution-final.txt`): 48 follow coder A, 16 follow coder B and one takes a third value (the granularity of S01). Three units once flagged as pending or conflicting (the mechanism of S18, S19 and S68) were checked against the source papers and confirmed. The disagreement list is `reconciliation-sheet.csv`, the first two rounds are `reconciliation-resolution-1-full.csv` and `reconciliation-resolution-2.csv`, the third-round package is `adjudication-round3-guide.md`, and the merge script is `coding/kappa/merge_recon`.

## 4 S4. The 121-row coding file and the historical audit

- All 121 rows carry category, granularity, mechanism and object type (`coding-121-final.csv`; the Appendix mirror is `audit/appendix-matrix-code.csv`);
- The rule-based coding of the single-coder stage and its difference list (`audit/code_matrix.py`, `internal-audit.md`) are kept as a historical trace. At that stage 23 of the 121 rows were changed from the default rule by S-level overrides; after two-coder independent coding replaced it, the number of rows on which the same rule differs from the final coding rose to 87, which shows that rule-based coding cannot substitute for human judgement;
- The textual basis for the six most contestable entries (WELDER, HAQ, Checkmate, Stream, ODiMO and PowerFlow-DNN) is in `matrix/matrix-empty-cells.md`.

## 5 S5. Independent spot-check protocol (optional)

If a journal or reviewer asks for a non-author check: a third party fills the independent columns of `audit/coding-audit-sample30.csv` following `audit/coding-audit-protocol.md`, then runs `audit/audit_coding.py` to obtain the agreement rate and Cohen's  $\kappa$ .

## 6 S6. Small-scale validation experiment

For the experiment reported in the small-scale validation experiment section, the measurement protocol, the raw per-run measurements, the configuration matrix, the job package and the plotting scripts are all under `experiment/`: the description is `experiment/README.md`, the environment and acceptance commands are `experiment/ENV_AND_EXPERIMENT.md`, the per-scan result tables and summaries are under `experiment/data/` together with the archived per-run logs, the job package is `experiment/job/`, and the plotting scripts are `experiment/scripts/`. The complete configuration matrix of the second batch is also given directly as `experiment/batch2_5models.results.csv`.

## 7 S7. Post-window tracking and reproduction environment

The list of preprints tracked after the May 2026 search window (MileStone, AutoPass, DualScale) is `methodology/2026-preprint-tracking.md`; the environment and acceptance commands needed for the second experiment batch are `experiment/ENV_REQUIREMENTS.md`.

### B. Version and verification status

- Main text: continuous XeLaTeX build passes with no undefined references;
- Coding: `coding/kappa/merge_reconciliation.py`, `matrix/count_matrix.py` and `coding/kappa/sync_ap` can be re-run;
- Consistency audit: `audit/audit_scope_appendix_consistency.py` reports no inconsistencies found, and `audit/crosscheck_appendix.py` reports 0 mismatches between the sample table and its machine-readable mirror;
- Coders: two coders coded independently, followed by two reconciliation rounds and a third joint resolution round; a non-author spot check has not yet been carried out (S5 is an optional strengthening step).